



Projet 16

EY & Associés – Forensic & Integrity

Création automatique de graphes de relations à partir
d'un rapport de mission

Akram Bougoffa, Haïtem El Harras, Cyril Labeye, Hannah Laib, Rémy Maurice-Demourieux.

Année 2025-2026

SOMMAIRE

1/ Contexte du projet.....	3
1.1 L'entreprise commanditaire : EY & Associés.....	3
1.2 Le département métier : Forensic & Integrity Services.....	3
1.3 La problématique opérationnelle.....	3
2/ Objectifs du projet.....	3
3/ Développement.....	4
3.1 Structure du code.....	4
3.1.1 Le découpage et la préparation des données.....	5
3.1.2 L'extraction et la normalisation.....	5
3.1.3 La validation et fusion.....	7
3.1.4 Interface.....	8
3.2 Module de test.....	9
4/ Résultats.....	10
4.1 Résultats obtenus.....	10
4.1.1 Premiers résultats.....	10
4.1.2 Améliorations de l'affichage.....	11
4.2 Analyse statistiques.....	13
5/ Limites.....	16
5.1 Limites tokens et API.....	16
5.2 Limites de temps.....	16
5.3 Limites de l'IA.....	17
6/ Méthodologie et organisation.....	17
6.1 Planning du projet.....	17
6.1.1 Méthode.....	17
6.1.2 Organisation.....	17
7/ Conclusion.....	20
8/ Bibliographie.....	20

1/ Contexte du projet

1.1 L'entreprise commanditaire : EY & Associés

Notre projet provient de l'entreprise **EY (Ernst & Young)**, l'un des cabinets d'audit et de conseil les plus renommés. EY accompagne des entreprises internationales, des gouvernements et des institutions publiques sur des problématiques variées : audit financier, fiscalité, transactions et conseil en stratégie.

1.2 Le département métier : Forensic & Integrity Services

Au sein d'EY, notre projet est proposé par le **département Forensic & Integrity Services**. Ce service est spécialisé dans l'investigation numérique, la lutte contre la fraude, la conformité et la gestion de crise.

Ils agissent comme des "détectives financiers". Ils interviennent notamment pour :

- Mener des enquêtes internes suite à des soupçons de détournements ou de corruption.
- Réaliser des "Background Checks" (vérifications d'antécédents) sur des personnes avant une fusion-acquisition ou un partenariat stratégique.
- Cartographier des réseaux d'influence complexes.

1.3 La problématique opérationnelle

Dans l'exercice de leurs missions, les équipes produisent systématiquement des rapports d'analyse. Ces documents détaillent les liens identifiés entre différentes entités : liens capitalistiques (qui détient qui), liens de parenté, mandats sociaux (qui dirige quoi), et flux financiers.

Actuellement, pour visualiser ces connexions et détecter d'éventuels conflits d'intérêts, les équipes doivent :

- Lire intégralement le rapport.
- Extraire manuellement les informations.
- Dessiner le graphe de relations "à la main" ou via des outils basiques.

Ce processus manuel est fastidieux, chronophage et source d'erreurs potentielles (oubli d'un lien, mauvaise interprétation).

2/ Objectifs du projet

Le but de ce projet est d'automatiser la création de graphes de relations à partir d'un ensemble de textes, quelle que soit leur nature (rapports de mission, articles, documents institutionnels, etc.). L'objectif est de produire des graphes permettant de visualiser et d'analyser les relations morales ou physiques existantes entre les différentes entités mentionnées dans les textes, qu'il s'agisse de personnes, d'entreprises, d'organisations ou d'institutions.

Pour ce faire, le projet s'appuie sur l'intelligence artificielle générative afin d'extraire automatiquement des données pertinentes à partir de textes non structurés. Cette extraction devra permettre d'identifier les entités présentes dans le texte, puis d'établir les relations qui existent entre elles (collaboration, hiérarchie, interaction, dépendance, etc.). La qualité et la précision de cette étape sont essentielles, car elles conditionnent directement la fiabilité des graphes produits.

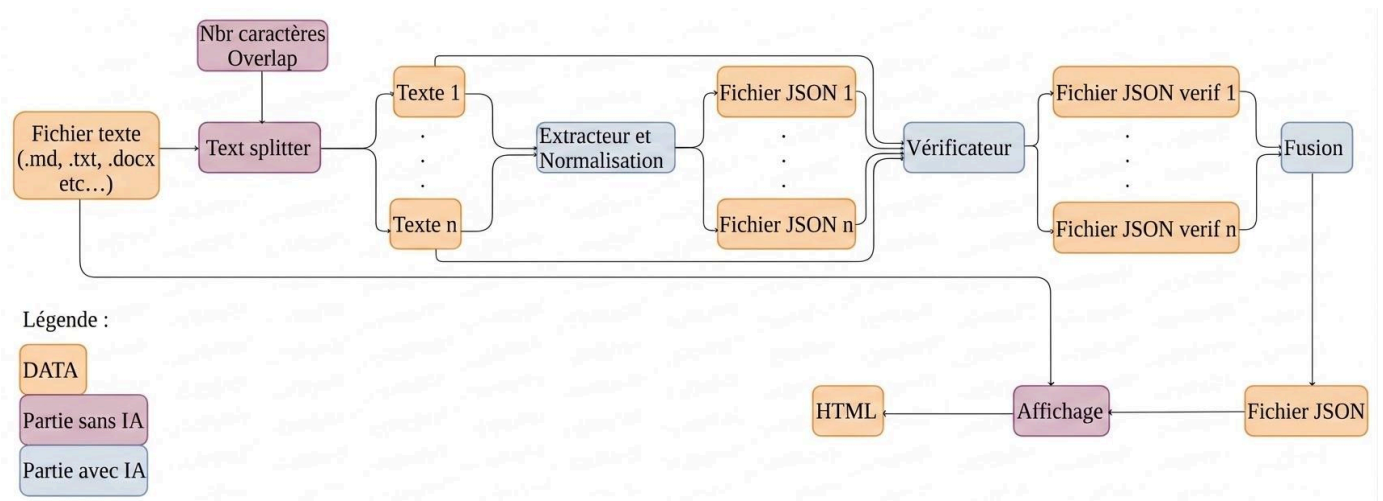
Ce sujet est particulièrement d'actualité, en raison de l'augmentation de l'utilisation de l'intelligence artificielle dans les entreprises et dans l'analyse et l'exploitation des données textuelles.

3/ Développement

Le développement sera réalisé exclusivement en Python. Pour la réalisation de ce projet, plusieurs bibliothèques ont été utilisées afin de structurer et automatiser le traitement des données.

3.1 Structure du code

La structure globale du programme repose sur une architecture en cinq phases principales : le découpage du document, l'extraction/normalisation structurée, la validation avec boucle de correction et enfin la fusion des données et l'affichage interactif. L'objectif est de transformer des documents textuels non structurés (au format Markdown) en un graphe de connaissances standardisé, propre et visuellement exploitable.



3.1.1 Le découpage et la préparation des données

Avant toute interaction avec le modèle de langage, le document brut subit une phase de préparation essentielle réalisée sans intelligence artificielle. Cette étape préliminaire vise à transformer un rapport textuel volumineux en données exploitables par le système. En effet, les modèles de langage possèdent des limites de contexte : il n'est pas possible d'envoyer

un rapport entier contenant plusieurs dizaines de pages sans risquer une perte d'informations, une diminution de la qualité des réponses ou encore des incohérences dans l'extraction des données. Afin de résoudre ce problème, le programme met en œuvre une stratégie de chunking, c'est-à-dire un découpage progressif du texte en segments plus petits et plus faciles à traiter.

Cette opération repose sur la bibliothèque *LangChain*, qui constitue un composant central de l'architecture logicielle du projet. *LangChain* permet de structurer les différentes étapes de traitement des données textuelles et d'automatiser l'enchaînement des opérations liées à l'analyse documentaire. Dans le cadre de cette première phase, le système exploite plus précisément le module *RecursiveCharacterTextSplitter* fourni par *LangChain*. Cet outil réalise un découpage intelligent du texte en conservant la continuité des informations importantes.

Le fonctionnement adopté consiste à segmenter le rapport en blocs d'environ 12 000 caractères avec un chevauchement de 400 caractères entre chaque segment. Ces valeurs ont été déterminées arbitrairement afin d'obtenir un compromis satisfaisant entre quantité d'informations transmises au modèle et stabilité des résultats obtenus. Le chevauchement joue ici un rôle fondamental. Sans cela, certaines entités ou relations présentes entre deux segments pourraient être coupées et donc partiellement perdues lors de l'analyse.

Cette phase de préparation représente également une optimisation importante des performances globales du programme. En effet, le traitement par blocs permet de paralléliser certaines opérations, de réduire les coûts d'inférence du modèle et d'améliorer la stabilité des extractions. De plus, le découpage facilite la reprise du traitement en cas d'erreur : si un segment échoue, il n'est pas nécessaire de traiter de nouveau l'ensemble du document.

3.1.2 L'extraction et la normalisation

Une fois les données préparées et segmentées, le système entre dans sa phase principale : l'extraction des informations et leur normalisation. Contrairement à une approche classique où ces deux opérations seraient réalisées séparément, notre programme les combine dans un même processus afin d'améliorer la cohérence globale des données produites.

Le cœur de cette étape repose à nouveau sur la bibliothèque *LangChain*, utilisée ici pour orchestrer les interactions avec le modèle de langage et formaliser les instructions transmises au système d'intelligence artificielle. Cela permet notamment de structurer les prompts, de définir des rôles métiers précis et de gérer automatiquement les échanges avec le modèle utilisé.

Le modèle choisi pour l'extraction est *gemini-3.1-flash-lite-preview*, utilisé via *ChatGoogleGenerativeAI* intégré dans *LangChain*.

Pour guider le modèle, le prompt de génération suit une structure rigoureuse inspirée de la méthode **CRAFT** :

- **Contexte** : Il s'agit de définir le cadre dans lequel s'inscrit la tâche. Dans notre cas, le

modèle est informé qu'il doit analyser un document dans le but d'en extraire des informations spécifiques, telles que des entités (personnes, entreprises) et des relations. Ici nous utilisons une méthode de chunking qui consiste à ne pas envoyer tout le rapport d'un coup, mais le traiter page par page.

- **Rôle** : On attribue au modèle une identité ou une expertise particulière afin d'orienter son comportement.
- **Action** : Cette partie définit précisément ce que le modèle doit faire. Dans le cadre du projet, il s'agit d'identifier les entités (personnes, organisations), détecter les relations entre ces entités et extraire les informations pertinentes.
- **Forme** : Le modèle doit produire une sortie dans un format structuré et exploitable de type JSON. Cela permet d'obtenir une exploitation automatique des résultats et une intégration facile dans les étapes suivantes (graphe, analyse).
- **Ton** : Le ton précise la manière dont le modèle doit formuler sa réponse. Dans ce projet, le ton est neutre, précis et factuel. Nous avons testé plusieurs prompts et cette méthode permet de formuler des instructions claires, précises et reproductibles afin d'obtenir des résultats fiables.

Cependant, l'extraction seule ne suffit pas pour construire un graphe relationnel exploitable. Les données textuelles présentent naturellement des variations de forme qui peuvent provoquer des incohérences importantes. Une même personne peut être mentionnée sous différentes écritures, par exemple "*Alain Dupont*", "*A. Dupont*" ou encore "*Monsieur Dupont*". Sans traitement complémentaire, ces variations généreraient plusieurs nœuds distincts dans le graphe final.

Afin de résoudre ce problème, le programme implémente une phase de normalisation s'appuyant sur la bibliothèque *Pydantic* et sur une démarche programmation objet. Pour ce faire, nous avons structuré nos données en utilisant des classes Python qui héritent de la classe `BaseModel` de *Pydantic*. Cette classe permet de donner les instructions à l'IA pour un attribut spécifique. Cela permet de définir des schémas de données stricts et d'imposer une structure cohérente aux informations extraites. Les entités, relations et attributs sont ainsi validés automatiquement avant leur intégration dans le système.

Exemple ci-dessous de classe :

```
class Lien(BaseModel):
    source: str = Field(description="Exact ID of the source entity. Must strictly match the 'id' attribute of a previously defined object in the 'entities' list to guarantee referential integrity.")
    target: str = Field(description="Exact ID of the target entity. Must strictly match the 'id' attribute of a previously defined object in the 'entities' list.")
    relation: str = Field(description="Descriptive label of the connection. Mandatory format: lowercase snake_case, purely ASCII with no accents or special characters (e.g., 'ceo_of', 'majority_shareholder', 'associated_with').")
```

```
gravite: int = Field(ge=1, le=5, description="Criticality level of the interaction (1 = weak, indirect, or historical link; 5 = direct involvement, financial tie, or strong hierarchical control).")
```

Tout d'abord, les types d'entités sont standardisés : chaque entité extraite doit appartenir à une catégorie prédéfinie telle que *“personne”*, *“organisation”* ou *“institution”*. Ensuite, les relations sont elles aussi normalisées grâce à un vocabulaire contrôlé. Si le modèle produit des formulations proches ou synonymes, celles-ci sont automatiquement ramenées vers une relation standard unique.

Le système effectue également un nettoyage des incohérences de formatage. Certaines réponses du modèle peuvent contenir des erreurs structurelles, comme l'utilisation d'une chaîne de caractères à la place d'une liste, par exemple "Hugo" au lieu de ["Hugo"]. *Pydantic* permet de corriger ou de détecter automatiquement ces anomalies.

Enfin, un mécanisme de gestion des alias assure la fusion des variantes de noms associées à une même entité. Cette fonctionnalité est essentielle pour garantir l'unicité des données dans le graphe final.

Sans cette étape de normalisation, le graphe généré serait fragmenté et difficilement exploitable. Plusieurs nœuds représenteraient la même personne ou organisation, tandis que les relations seraient multipliées sous des formulations différentes. La qualité analytique du graphe serait alors fortement dégradée. Cette phase constitue donc un élément central du programme, car la cohérence des données extraites conditionne directement la pertinence de la visualisation relationnelle finale.

3.1.3 La validation et fusion

Après la génération du premier fichier JSON par l'agent d'extraction, les données sont transmises à un nœud de vérification chargé d'évaluer leur cohérence et leur fidélité par rapport au texte source. Cette architecture est organisée grâce à la bibliothèque *LangGraph*, qui constitue une extension de *LangChain* dédiée à la modélisation sous forme de graphes.

Dans notre système, *SpaCy* est utilisée comme couche de validation complémentaire afin de vérifier les résultats produits par le modèle de langage. Cette bibliothèque est spécialisée en traitement automatique du langage naturel et offre des fonctionnalités telles que la reconnaissance d'entités nommées. Elle permet de compléter et de vérifier les résultats obtenus par le modèle de langage, tout en améliorant la précision de l'extraction. Son utilisation apporte ainsi une couche supplémentaire de validation des données. L'objectif est d'introduire une vérification plus déterministe et moins dépendante d'un LLM. Cette approche permet d'améliorer la robustesse du système.

L'agent de validation agit ainsi comme un auditeur de données. Il compare les entités détectées par le modèle avec celles identifiées par *SpaCy* dans le texte original. Les incohérences potentielles peuvent alors être signalées ou corrigées avant l'intégration finale des données.

Une fois l'ensemble des segments du document analysés et validés, le système dispose d'une série de fichiers JSON partiels correspondant à chaque portion du texte traité. La phase de fusion vise alors à regrouper ces données fragmentées dans une structure unique cohérente.

Cette opération est également orchestrée grâce à *LangGraph*, qui facilite la circulation des données entre les différentes étapes du pipeline. Le système analyse les chevauchements entre segments afin d'identifier les entités présentes dans plusieurs blocs différents.

Les alias et pseudonymes détectés au cours de l'analyse sont alors consolidés afin de garantir qu'une seule entité finale soit conservée dans le document maître. Les relations sont elles aussi fusionnées et dédoublées afin d'éviter les répétitions inutiles.

Cette phase permet donc de reconstruire progressivement une représentation globale cohérente du rapport analysé malgré le traitement fragmenté imposé par le chunking.

3.1.4 Interface

Afin de rendre notre architecture logicielle accessible et directement exploitable par les équipes d'audit, nous avons développé une application web interactive. Notre choix technologique s'est porté sur la bibliothèque *Streamlit*. Elle est particulièrement performante pour le déploiement rapide d'interfaces orientées Data Science.

L'interface a été conçue autour de principes d'ergonomie et de minimalisme, afin de rendre l'outil directement exploitable sans nécessiter de compétences techniques préalables. Le parcours utilisateur se décompose en trois étapes algorithmiques :

1. L'ingestion sécurisée des données : L'utilisateur est invité à déposer son rapport (au format Markdown) via une zone de dépôt de fichier (`file_uploader`). Le script Python récupère alors le buffer de ce fichier et le sauvegarde de manière temporaire sur le serveur local (`temp_fichier.md`) afin de le rendre disponible pour la phase de découpage (chunking).

2. La gestion du traitement asynchrone : L'extraction algorithmique, orchestrée par *LangGraph* et impliquant de multiples requêtes vers l'API Gemini, est un processus coûteux en temps. Pour pallier ce problème d'expérience utilisateur et éviter que l'auditeur ne pense que l'application est bloquée, nous avons implémenté un système de notifications d'état dynamiques. Des composants visuels (comme `st.info` et `st.success`) informent l'utilisateur en temps réel de l'avancement de la création du graphe de connaissances.

3. Le rendu dynamique du graphe : L'affichage final représentait un défi technique spécifique. En effet, la bibliothèque de visualisation *Pyvis* produit des fichiers HTML autonomes (contenant du code JavaScript complexe pour la physique des nœuds), qui ne s'intègrent pas naturellement dans les conteneurs standards de *Streamlit*. Pour contourner cette limitation logicielle, nous avons fait appel au module `streamlit.components.v1`. Cette solution nous a permis d'encapsuler la chaîne de caractères HTML générée par *Pyvis* et de l'afficher directement au sein d'une balise `iframe` (`components.html`) dans notre application.



Ainsi, l'utilisateur final dispose d'un environnement unique et fluide pour passer d'un document textuel brut à un graphe interactif, avec la possibilité de zoomer et de manipuler les sommets directement depuis son navigateur web.

3.2 Module de test

Pour évaluer la précision de notre outil d'extraction, nous avons un problème : *si on donne un vrai rapport Enron à l'IA, comment mesurer son taux d'erreur, puisqu'on ne connaît pas nous-mêmes toutes les entités du rapport ?*

Nous avons donc créé un générateur de données sur mesure. Le principe est simple : on génère un texte dont on connaît déjà toutes les relations exactes, et on vérifie si l'IA réussit à faire un sans-faute, c'est-à-dire reconstituer le graphe initial. La démarche se déroule en quatre étapes :

1. Création d'un réseau de référence

Le programme commence par générer automatiquement un réseau de relations artificielles. Il sélectionne un certain nombre d'entités (personnes et entreprises) et crée des liens entre eux (relations familiales, professionnelles, etc.). L'algorithme s'assure de la logique et de la réciprocité de chaque lien : par exemple, si un acteur est défini comme l'employeur d'un autre, le second devient logiquement son employé.

2. Textualisation et ajout d'informations parasites

Pour reproduire la complexité d'un vrai rapport, ces données brutes sont transformées en un document textuel grâce à une liste de phrases types variées. Afin de tester la solidité de notre système face à des informations inutiles, nous ajoutons volontairement des données parasites :

- Un bruit sans lien : des phrases aléatoires totalement hors sujet qui ne contiennent pas de relations parasites.

- Un bruit avec lien : un texte narratif continu (la légende du Roi Arthur). Ce texte sert de "piège" pour vérifier si le modèle va confondre les personnages du récit avec les entités du graphe initial.

3. Comparaison et analyse du sens

Une fois ce document analysé par notre programme, le résultat produit par l'IA est comparé à notre réseau de départ. Puisque l'intelligence artificielle reformule souvent les termes, une simple comparaison mot pour mot serait inefficace.

Pour les entités (les acteurs) : Nous appliquons un nettoyage du texte (gestion des accents et des majuscules) et utilisons une liste de synonymes. Cela nous permet de compter avec précision les acteurs inventés (faux positifs) et les acteurs oubliés (faux négatifs).

Pour les relations (les liens) : Nous utilisons la bibliothèque *spaCy* pour évaluer le sens des mots. Grâce à cet outil, si l'IA extrait la relation "*dirigeant de*" au lieu de "*directeur de*", le système comprend que la signification est très proche et valide la relation.

4. Automatisation et mesure des performances

L'ensemble de ce processus fonctionne en boucle. Le script est capable de générer, perturber et évaluer des dizaines de graphes à la suite sans intervention humaine. Les taux d'erreurs sont ensuite sauvegardés et moyennés. Cela nous permet de calculer avec fiabilité le score de précision global de notre application et de mesurer l'impact direct de chaque modification apportée à notre code.

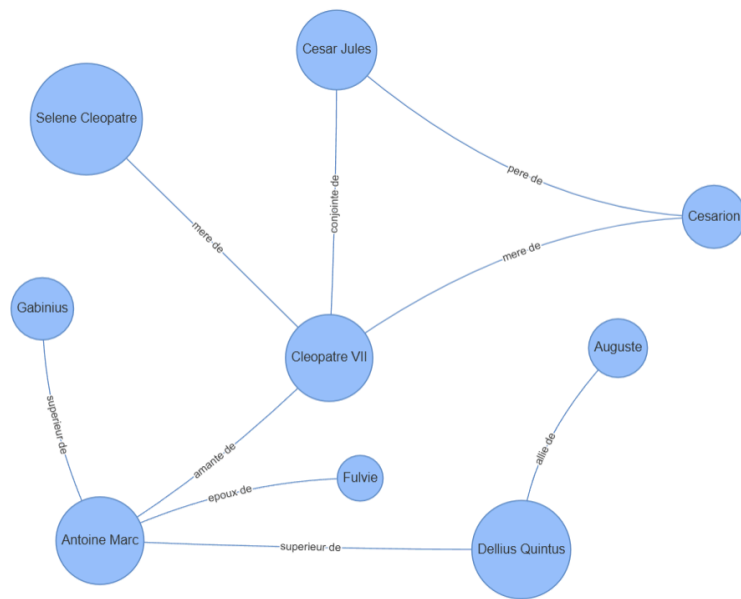
Nous traiterons les statistiques obtenues par ce module dans la **section 4.1.2**

4/ Résultats

4.1 Résultats obtenus

4.1.1 Premiers résultats

Dans un premier temps nous avons obtenu le graphe ci-dessous (testé sur un extrait de la page Wikipédia de Cléopâtre).



Cette première version de notre projet constitue une base fonctionnelle satisfaisante. Nous avons bien réussi à modéliser nos entités (qui se limitent ici uniquement à des personnes) sous forme de sommets, et à représenter leurs interactions par des segments de liaison. Cependant, en l'état, ce graphe présente encore des limites que nous avons prévu de corriger dans notre prochaine version.

Voici les principaux axes d'amélioration sur lesquels nous allons travailler :

- **La différenciation par type d'entité** : Actuellement, le graphe est uniforme. L'objectif est d'intégrer un code couleur spécifique afin de distinguer instantanément les différentes entités. Cela facilitera grandement la lecture et l'analyse globale.
- **Un redimensionnement des sommets** : Dans cette version initiale, la taille des nœuds est simplement définie par la longueur de la chaîne de caractères (le nom de l'entité). Cette approche manque d'intérêt analytique. Nous allons donc lier la taille des sommets à une logique d'extraction mathématique, comme le nombre d'occurrences dans le texte source, afin de faire ressortir naturellement les acteurs centraux de l'affaire.

En appliquant ces modifications, nous pourrions transformer ce premier schéma de base en une visualisation beaucoup plus claire, pertinente et agréable à exploiter pour comprendre les enjeux du document.

4.1.2 Améliorations de l'affichage

Pour le cas d'étude Enron, nous avons implémenté l'ensemble des améliorations projetées lors de notre première itération. Le graphe final présente une évolution technique majeure, transformant un simple schéma relationnel en un véritable outil d'analyse visuelle.

4.2 Analyse statistique

Concernant la comparaison des sommets, nous avons d'abord comparé les identifiants (ID), puis les alias. Cette vérification des alias est indispensable : selon la façon dont le modèle extrait les informations, une même personne peut se retrouver avec des identifiants différents. Nous nous sommes d'ailleurs demandé s'il fallait inverser les noms et prénoms pour chaque sommet. Finalement, comme nous imposons un format de sortie strict à notre LLM, il nous a suffi d'adapter notre jeu de données à ce format là.

Pour la comparaison sur les liens, cela a été un peu plus compliqué. Nous avons exploré plusieurs pistes. C'est dû au fait que nous ne comparons plus des noms d'entités morales ou physiques, mais des phrases. L'une des premières pistes envisagées a été de faire appel à un LLM, mais nous nous sommes dit qu'une version plus déterministe serait plus adéquate. En effet, nous aurions eu le même problème que sur notre application : comment s'assurer que la valeur renvoyée est correcte ? Une fois que la méthode déterministe a été choisie, nous avons envisagé plusieurs pistes. L'une des premières a été d'utiliser `"spacy.load('fr_core_news_md')"`, qui sert à analyser la structure d'une phrase. Mais nous nous sommes aperçus que la détection des informations n'était pas correcte. En effet, « *est le fils de* » et « *est le fils illégitime de* » relèvent de la même information ; bien qu'il y ait une perte d'information pour l'une, l'idée reste la même. À cette perte d'information, le module *spaCy* nous renvoyait un faible taux, ce qui mettait le lien dans les "perdus" alors qu'il était juste. Une fois que nous avons pris conscience de cela, nous nous sommes alors tournés vers des modèles spécialisés dans la comparaison sémantique de phrases, comme les *Sentence Transformers*, mieux adaptés à ce type de nuance."

Bien que les *Sentence Transformers* fassent partie de la même famille que les LLM, leurs rôles sont diamétralement opposés. En effet, là où un LLM génère du texte en guise de réponse, un *Sentence Transformer* va convertir l'information en vecteurs numériques. De plus, contrairement aux LLM dont les réponses peuvent varier, les *Sentence Transformers* sont totalement déterministes : le résultat obtenu pour une phrase donnée sera toujours strictement le même. On pourrait comparer cette méthode à un bibliothécaire qui rangeait ses livres selon leurs thèmes ou par ordre alphabétique. Ainsi, pour comparer deux phrases, l'algorithme observe à quel point leurs vecteurs sont proches. C'est ce que l'on appelle la similarité cosinus, une métrique mathématique qui évalue la proximité entre deux vecteurs. Une fois ce taux obtenu, on le compare à un seuil défini au préalable. Si ce taux est supérieur ou égal au seuil, on incrémente une variable temporaire.

Une fois toutes ces comparaisons terminées, nous appliquons les formules suivantes afin de mener à bien notre analyse statistique :

```
# statistiques sur les sommets
len_d = max(len(list_depart), 1)
stat_faux_pos_s = round(len(manquants) / len_d * 100, 3)
stat_faux_neg_s = round(len(inventes) / len_d * 100, 3)

# statistiques sur les liens
len_d = max(len(list_dep_lien), 1)
```

```
stat_faux_pos_l = round((len(list_dep_lien) / 2 - nbre_resultat) /  
len_d * 100, 3)  
stat_faux_neg_l = round((len(list_arriver_dicts) / 2 -  
nbre_resultat_inv) / len_d * 100, 3)
```

Nous constatons une division par deux. Cela s'explique par le fait que les informations sont stockées dans les deux sens. Ensuite, à la taille totale de la liste, nous soustrayons la variable temporaire « `nbre_resultat` » (définie plus haut), qui correspond aux éléments non trouvés ou mal identifiés.

Voici une analyse des statistiques obtenues :

```
Analyse des statistiques (112 test(s))  
ENTITÉS (Sommets) :  
- Inventés (Faux Positifs) : 34.749 %  
- Oubliés (Faux Négatifs) : 3.692 %  
  
RELATIONS (Liens) :  
- Inventées (Faux Positifs) : 39.874 %  
- Oubliées (Faux Négatifs) : 22.69 %  
  
SCORE GLOBAL :  
- Précision globale du graphe : 74.751 %
```

Nous avons un jeu de tests composé de 112 tests, pour lesquels le nombre de sommets a été fixé à 15. Nous aurions aimé disposer d'un plus large panel de tests, mais nous avons mis du temps à trouver la comparaison adéquate pour les liens. De plus, comme nous utilisons une API gratuite, nous sommes limités par le nombre de requêtes quotidiennes.

L'analyse de nos statistiques montre que les résultats sont compris entre les valeurs 64,189 et 83,294. Cependant, ils sont extrêmement concentrés autour de 74,751 % en ce qui concerne la précision globale du graphe.

Concernant les sommets (entités), nous remarquons que notre modèle est assez fiable sur la notion de faux négatifs. En effet, il comptabilise en moyenne 3,692 % de sommets oubliés, ce qui représente moins d'un sommet par test. Le pourcentage d'erreurs varie entre 0.0 et 35,135. Cette différence d'amplitude peut être due au type de données ; en effet, dans nos textes, il peut y avoir des noms de famille qui se ressemblent.

Pour ce qui est des faux positifs, on remarque que le taux est nettement supérieur à celui des faux négatifs. On peut en déduire que notre modèle a davantage tendance à halluciner. Lors de la phase d'analyse, nous avons eu accès aux données brutes et nous avons pu constater la chose suivante : les sommets hallucinés étaient souvent une composition possible à partir des noms et prénoms présents dans notre jeu de tests.

La question que l'on se pose est la suivante : est-ce que le fait de lancer le LLM en boucle l'empêche de distinguer qu'il s'agit d'un nouveau jeu de tests, l'amenant ainsi à prendre en considération les textes précédents et à mélanger les informations ?

Concernant l'extraction des liens, nous remarquons que cette tâche s'est avérée beaucoup plus complexe que l'identification des sommets. En effet, elle représente environ 62 % de l'imprécision globale de notre projet.

Nous observons un taux de faux positifs très élevé, frôlant les 40 % (39,874 %). Cela indique que le modèle a une forte tendance à halluciner en inventant des connexions qui n'existent pas dans le texte source. Ce fort taux d'erreur peut s'expliquer par trois sources d'erreurs possibles. Tout d'abord, si l'un des deux sommets n'est pas détecté et est absent de notre jeu de données, le lien est directement comptabilisé comme inventé (faux positif), sans même que sa nature soit testée. Ensuite, une fois la vérification des sommets effectuée, si la correspondance des liens est inférieure à 75 %, ils sont également basculés dans la catégorie des liens inventés.

En parallèle, le taux de faux négatifs s'élève à 22,69 %. À l'inverse des faux positifs, nous ne pouvons pas attribuer cette erreur à une mauvaise détection des sommets (à part s'il s'agit du sommet central de notre jeu de tests). Ce taux peut être dû au fait que, si les informations ne se trouvent pas les unes à la suite des autres dans le texte, le LLM a du mal à faire le lien, allant jusqu'à rater complètement leur existence. De plus, si la syntaxe de la phrase est complexe (voix passive, multiples propositions subordonnées, figures de style), le LLM aura des difficultés à raisonner comme un être humain et donc à identifier ces relations.

En conclusion de notre analyse statistique, le LLM parvient bien à identifier les acteurs, mais peine considérablement à comprendre et à retranscrire la sémantique exacte qui les relie.

Voici quelques erreurs que nous avons rencontrées :

Sur les sommets :

- La gestion des accents : par exemple, "Haïtem" devient "Haitem". Cette mauvaise gestion entraîne une confusion, car le modèle peut considérer "Haïtem" et "Haitem" comme deux personnes différentes alors qu'il s'agit du même individu, ou à l'inverse, considérer deux personnes distinctes comme une seule et même entité si leurs noms ne diffèrent que par un accent.
- La gestion des titres de noblesse : ils sont considérés comme des noms de famille par exemple, "*le Roi Arthur*" est interprété comme une seule entité dont "*le Roi*" ferait partie du nom, au lieu d'être reconnu comme un titre séparé du prénom "*Arthur*".

Sur les liens :

- Une perte d'information pouvant entraîner une mauvaise détection de liens
- Une perte d'information liée au bruit contenu dans le texte

Pour finir, lorsque le module de test termine son exécution, il affiche plusieurs moyennes globales dans le terminal. En effet, nous faisons en sorte de stocker les différentes informations de chaque simulation dans un fichier CSV. Le script calcule

ensuite la moyenne de l'ensemble des tests afin de nous fournir un pourcentage final représentatif de la fiabilité de notre application.

Ces résultats nous ont permis d'identifier les limites actuelles de notre application et constituent une base solide pour envisager des améliorations futures.

5/ Limites

5.1 Limites tokens et API

L'utilisation d'API externes pour interroger des modèles de langage a mis en évidence une limite de tokens. Cette contrainte impose une restriction sur la quantité de texte pouvant être envoyée et traitée en une seule requête. Dans le cadre de documents volumineux, cette limitation a nécessité la mise en place de stratégies de découpage des données, afin de respecter les contraintes imposées par l'API tout en conservant un maximum d'informations pertinentes.

Afin de contourner ces limitations et de réduire la dépendance aux services externes, une solution consistant à exécuter les modèles en local a été envisagée. Cependant, cette approche s'est révélée difficile à mettre en œuvre en raison des importantes ressources de calcul requises. L'exécution locale de modèles de langage performants nécessite en effet une puissance de calcul élevée, notamment en termes de mémoire ce qui dépasse nos moyens matériels.

Pour répondre à ces contraintes, une solution alternative a été adoptée en utilisant Google AI Studio, qui met à disposition un crédit gratuit. Cette option a permis d'accéder à des modèles performants via une API, tout en limitant les contraintes techniques et financières. Elle avait cependant une certaine limite, un plafond étant donné qu'il ne s'agissait pas d'argent réel. Cette limite était suffisante pour lancer un test, mais pas pour permettre d'en enchaîner plusieurs, nous avons donc, encore une fois été contraints dans nos utilisations, on a pu essayer de limiter au maximum les appels au LLM, via une rectification de nos codes, en essayant de l'utiliser le moins possible.

5.2 Limites de temps

L'intervalle de temps entre le début du projet et la fin étant assez court, cela nous a contraints dans la qualité des fonctionnalités qu'on a pu proposer. Chaque test étant assez chronophage, il nous fallait à chaque fois attendre entre chaque modification pour voir si cela avait réellement un impact.

Plusieurs idées de fonctionnalités n'ont pas pu être développées ou implémentées, parmi celles-ci on peut lister :

- L'ajout d'une option de recherche dans le graphe que ce soit selon le nom, une sélection.
- La modification de la fusion des JSON en un algorithme Python plutôt qu'un appel LLM.

5.3 Limites de l'IA

Les IA génératives étant une technologie assez récente, surtout pour le grand public, présentent un défaut majeur : le manque de fiabilité. Nous ne pouvons savoir à l'avance le raisonnement utilisé par l'IA et il nous est donc impossible de prévoir totalement ce que produira cette dernière. Plus on utilise le LLM dans le code, moins l'on peut être sûr de notre résultat, c'est pour cela qu'il a fallu développer, dans la limite de ce qu'on a pu produire, des méthodes permettant de s'assurer de la qualité des données extraites par le LLM.

6/ Méthodologie et organisation

6.1 Planning du projet

6.1.1 Méthode

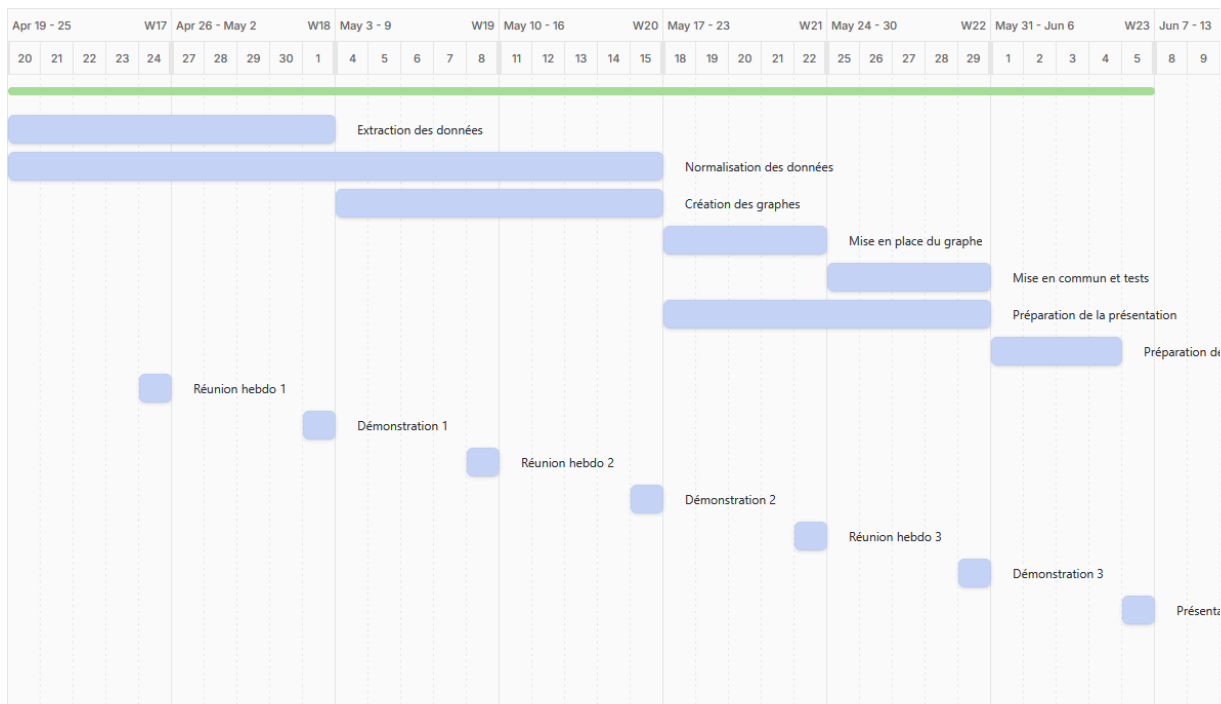
Pour réaliser ce projet, nous avons choisi d'utiliser la méthode "**Agile**" plutôt que la méthode en "V". En effet, en choisissant la méthode en V, il nous aurait été impossible de l'appliquer, étant débutants dans le domaine, nous n'aurions pas eu la possibilité d'anticiper les différents aléas du projet. Des connaissances plus approfondies en entrepreneuriat ainsi qu'une meilleure maîtrise des technologies auraient été nécessaires.

Par ailleurs, la méthode "Agile" nous permet de faire évoluer le projet de manière progressive tout en intégrant les retours du client. Cette méthode limite les risques de non-conformité et permet de livrer un projet plus proche des attentes réelles, notamment grâce à la collaboration entre les deux calls hebdomadaires.

Nous avons fait le choix de ClickUp pour représenter notre tableau de gestion selon la méthode Agile, car il est simple d'utilisation et permet une prise en main rapide.

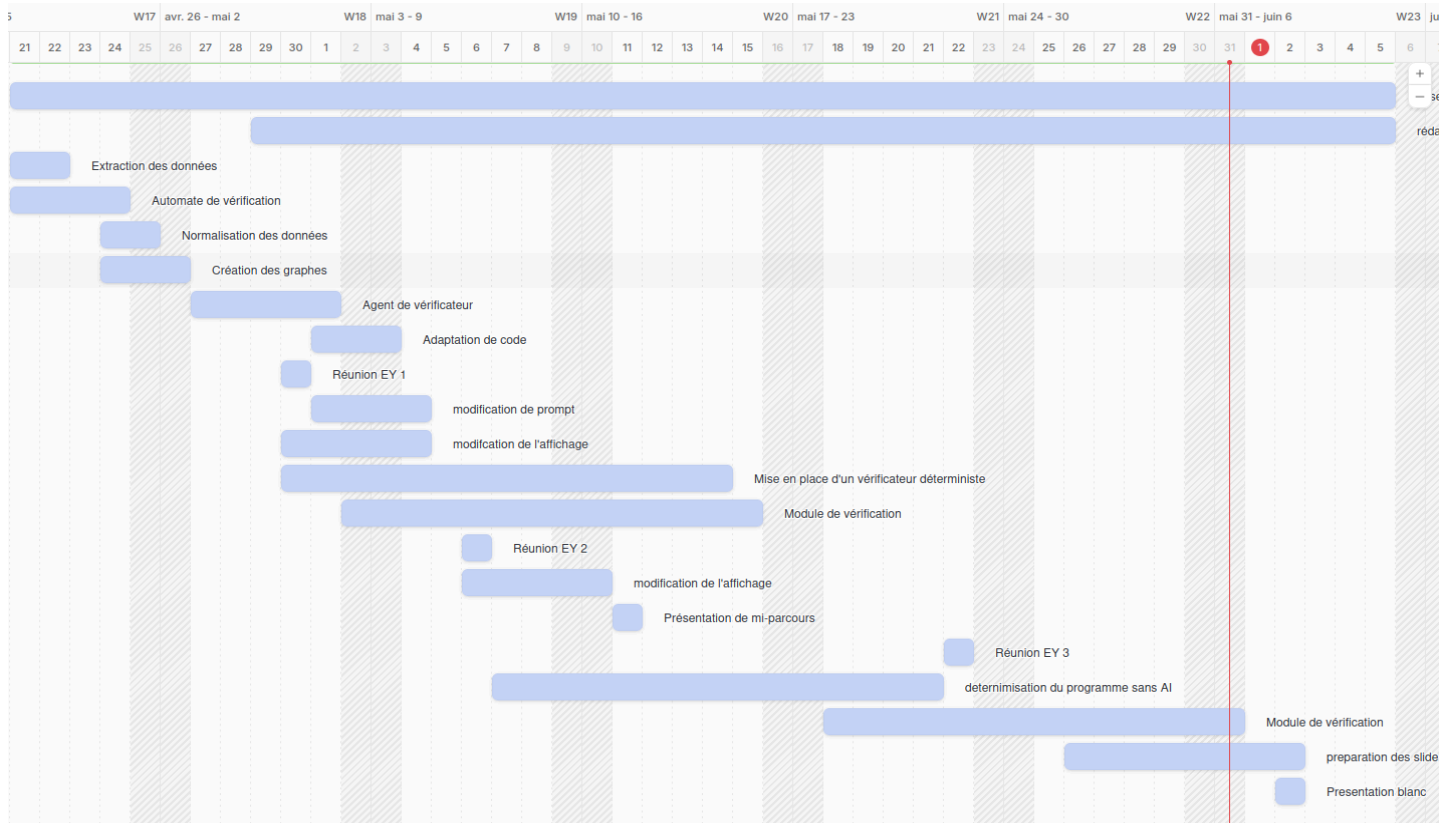
6.1.2 Organisation

Ci-joint notre diagramme de Gantt prévisionnel créé en décembre durant la première semaine de projet.



Le développement avait été structuré en trois grandes phases. Une première phase préparatoire permettant de définir l'architecture du programme ainsi que les technologies utilisées, notamment LangChain et LangGraph. La seconde phase, répartie sur sept semaines, devait être consacrée au développement du projet : création du module d'extraction et configuration des LLM, mise en place de la normalisation et de la gestion des états, puis conception et intégration du graphe interactif. Une semaine supplémentaire avait ensuite été dédiée à la fusion des différents modules afin d'obtenir une application cohérente et fonctionnelle. Enfin, la dernière phase devait porter sur les tests globaux, la correction des bugs et l'amélioration de la robustesse du système ainsi que de l'expérience utilisateur.

Cependant en commençant notre développement nous nous sommes rendu compte que la réalité différait beaucoup du plan que nous avons imaginé. Ce qui est normal vis-à-vis de notre manque d'expérience dans la réalisation de projet. Nous obtenons un diagramme de Gantt ci-dessous :



Phase de préparation des données : Le projet débute par l'Extraction des données qui s'étale sur environ 3 jours. Cette étape est immédiatement suivie par l'Automate de vérification (environ 3 jours), la Normalisation des données (environ 2 jours) et la Création des graphes (environ 3 jours), permettant de poser les bases techniques sur une période de 10 jours.

Phase de développement et itération : Cette période dense comprend l'Agent de vérificateur (environ 3 jours) et l'Adaptation de code (environ 2 jours). En parallèle, des tâches d'ajustement sont menées, notamment la modification de prompt (environ 4 jours) et une première modification de l'affichage (environ 4 jours). Le plus long est la mise en place d'un vérificateur déterministe, qui mobilise environ 10 jours, tandis que le Module de vérification principal s'étend sur une période globale d'environ 15 jours.

Phase de consolidation et finalisation : Après une Présentation de mi-parcours d'un jour, le projet se concentre sur l'affinage avec une seconde modification de l'affichage (environ 4 jours). La "déterminisation" du programme sans IA s'étale sur environ 12 jours, chevauchant la fin de la phase de développement. Enfin, la toute dernière étape de Module de vérification (environ 3 jours), la préparation des slides (environ 4 jours) et la Présentation blanche (1 jour) clôturent le projet.

7/ Conclusion

Pour conclure nous avons développé un outil automatisé en Python permettant de transformer des rapports textuels non structurés (format Markdown) en graphes de relations interactifs. Il repose sur plusieurs bibliothèques, pour le traitement des données, la normalisation, la validation des données et enfin l'affichage de notre graphe. Nous avons pu valider nos données pour renforcer la fiabilité des résultats. De plus, une interface utilisateur a été conçue pour faciliter l'utilisation de notre programme.

Durant ce projet nous avons rencontré des difficultés notamment dans la fiabilité de l'intelligence artificielle. C'est pour cela qu'un module de contrôle a été implémenté. Avec cet outil nous pouvons avoir des statistiques sur notre code afin de visualiser les potentiels axes d'améliorations pour affiner la marge d'erreur. Des limites liées à l'utilisation des clefs API nous ont freinés dans l'exécution de notre code, nous avons dû chercher des solutions pour pallier cela .

Nous avons également dû apprendre à bien coordonner notre développement car des méthodes différentes avaient été implémentées causant des problèmes de compatibilité.

Ce projet nous a donné l'opportunité de travailler avec l'intelligence artificielle, un outil particulièrement d'actualité dû à l'accroissement de son utilisation dans les entreprises et dans le monde en général ce que nous avons trouvé pertinent. Nous l'utilisons régulièrement mais dans le cadre de ce projet nous avons abordé ce sujet d'un point de vue différent ce qui nous a intéressé et motivé dans sa réalisation.

8/ Bibliographie

Bakst, W. (2025, 30 juin). A Guide to Prompt Templates in LangChain. Mirascope.
<https://mirascope.com/blog/langchain-prompt-template>

Constant, Agnès. « Le roi Arthur : histoire et légende. » Canoprof,
https://agnes-constant.canoprof.fr/eleve/Narration%20Rob%27Odyss%C3%A9e/leheros_corrige/res/Le_roi_Arthur_histoire_et_legende.pdf.

Contributeurs aux projets Wikimedia. *Affaire Sarkozy-Kadhafi*. 27 mai 2026,
fr.wikipedia.org/wiki/Affaire_Sarkozy-Kadhafi.

Contributeurs aux projets Wikimedia. *Cléopâtre VII*. 13 mai 2026, [fr.wikipedia.org/wiki/Cl % A9op % C3 % A2tre_VII](fr.wikipedia.org/wiki/Cl%C3%A9op%C3%A2tre_VII).

cyb31. « *Faire un cercle ou un point en HTML - Design*. » Communauté Jeedom, 5 juil. 2020, <https://community.jeedom.com/t/faire-un-cercle-ou-un-point-en-html/32546>.

Docstring. (s.d.). *Les sets en Python*. Docstring. <https://www.docstring.fr/glossaire/sets/>

« Documentation. » Pyvis 0.1.3.1 documentation, Read the Docs, <https://pyvis.readthedocs.io/en/latest/documentation.html>.

« *How to convert CSV to Excel with Python*. » YouTube, mis en ligne par PythonShinobi, 19 août 2025, <https://www.youtube.com/watch?v=vvqTITgK4is>.

marco056. « *Python : Retour ligne dans un csv*. » Developpez.net, 31 mars 2018, <https://www.developpez.net/forums/d1835994/autres-langages/python/general-python/retour-ligne-csv/>.

McKee, Amberle. « *Comment supprimer un fichier en Python*. » DataCamp, 14 nov. 2024, <https://www.datacamp.com/fr/tutorial/python-delete-file>.

Moreau, Philippe. « *Mise en page (les traits)*. » Université de Picardie Jules Verne, <https://www.u-picardie.fr/philippe/aideHTML/FormeTexte/Trait.html>.

pixegami. (2023, 18 septembre). *Pydantic Tutorial • Solving Python's Biggest Problem [Vidéo]*. YouTube. <https://www.youtube.com/watch?v=XIdQ6gO3Anc>

Pydantic. (s.d.). *Fields*. Pydantic. <https://pydantic.dev/docs/validation/latest/api/pydantic/fields/#pydantic.fields.Field>

Pydantic. (s.d.). *Models*. Pydantic. <https://pydantic.dev/docs/validation/latest/concepts/models/>

Pydantic. (s.d.). *Validators*. Pydantic. <https://pydantic.dev/docs/validation/latest/concepts/validators/>

« *Python Programming*. » YouTube, <https://www.youtube.com/@python-programming>.

. *Scandale Enron*. 9 mai 2026, fr.wikipedia.org/wiki/Scandale_Enron.

Shivanandhan, M. (2025, 5 novembre). *How to Use LangChain and LangGraph: A Beginner's Guide to AI Workflows*. freeCodeCamp. <https://www.freecodecamp.org/news/how-to-use-langchain-and-langgraph-a-beginners-guide-to-ai-workflows/>

Stats Wire. (2024, 16 février). *LangChain 08: Load PDF Document using Langchain| Python | LangChain [Vidéo]*. YouTube. <https://www.youtube.com/watch?v=Lol3lZJje9Y>

« *webbrowser — Convenient Web-browser controller*. » Python 3 Documentation, Python Software Foundation, <https://docs.python.org/3/library/webbrowser.html>.